Course Name:	**Database Management system Laboratory**	**Semester:**	**IV**
Date of Performance:	**17/02/2026**	**Batch No:**	**B2**
Faculty Name:	**Prof. Megha Sharma**	**Roll No:**	**16010124107**
Faculty Sign & Date:		**Grade/Marks:**	**25**

Experiment No: 3
Title: Implementation of Database in SQL -DDL

Aim and Objective of the Experiment:

Aim: Title: Implementation of Database in SQL -DDL
Objective: : Define/modify database definitions with proper constraints

COs to be achieved:

CO 2: Convert entity-relationship diagrams into relational tables, populate a relational database and formulate SQL queries on the data Use SQL for creation and query the database.

CO 3: Define and apply integrity constraints and improve database design using normalization techniques.

:

Books/Journals/Websites referred:

Books/ Journals/ Websites referred:

1. Sharaman Shah,"*Oracle for Professional*", SPD.
2. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g.Black book, Dreamtech Press
3. Korth, Slberchatz, Sudarshan: "Database Systems Concept", 5th Edition , McGraw Hill
4. Peter Rob and Carlos Coronel,"Database Systems Design, Implementation and Management", Thompson Learning, 5th Edition

Pre Lab/ Prior Concepts:
Resources used: Postgresql

Theory

Theory: The set of relations in a database must be specifies to the system by means of a data definition language (DDL). The SQL DDL allows specification of not only a set of relations but also specific information about the relation including,

1. The schema for each relation
2. The domain of values associated with each attribute
3. The integrity constraints
4. The set of indices to be maintained for each relation
5. The security and authorization information for each relation
6. The physical storage structure of each relation on disk

Syntax Create Table:

create table employee(ssn,fname varchar(10), mname varchar(10), lname varchar(10), desg

varchar(20), gender varchar(5), addr varchar(20), bdate datetime, sal float,primary key(ssn));

create table manages(ssn int, dept_code int, start_dt datetime, foreign key(ssn)

create table manages(ssn int, dept_code int, start_dt datetime, foreign key(ssn) references employee, foreign key(dept_code) references department, key(ssn,dept_code)) on delete set null;primary

Data Constraints

Business managers of the organization determine a set of rules that must be applied before the data is stored in the database. The application of such rules on raw data ensures **data integrity**.

Eg:- An employee belonging to the Sales department cannot have a salary higher than Rs. 1000. An employee has an unique identification number.

Applying Data Constraints

Oracle permits data constraints to be attached to table columns using SQL syntax. Constraints can be attached to table columns using Alter table

Unique Constraint

Unique Constraint- At column level Syntax

<ColumnName><Datatype>(<size>)

UNIQUE Unique Constraint- At table level

CREATE TABLE<TableName>(

<ColumnName><Datatype>(<size>)

<ColumnName><Datatype>(<size>)

<Columnname><Datatype>(<size>) UNIQUE(<ColumnName1>,<ColumnName2>);

Implementation Details (Problem Statement, Query and Screenshots of Results):

Data Output Messages Notifications

transaction_id [PK] integer	acc_no character varying (50)	txn_type character varying (10)	amt double precision	timestamp timestamp without time zone	payment_id integer

Data Output Messages Notifications

acc_no [PK] character varying (50)	balance double precision	created_at date	account_type character varying (20)	status character varying (20)	ifsc character varying (20)

Data Output Messages Notifications

ifsc [PK] character varying (20)	address character varying (200)

Data Output Messages Notifications

customer_id [PK] integer	name character varying (100)	email character varying (100)	phone_no character varying (15)	aadhar_no character varying (12)	acc_no character varying (50)	loan_id character varying (50)	allocation_id character varying (50)

Data Output Messages Notifications

emp_id [PK] integer	salary integer	hire_date date	name character varying (100)	department character varying (100)	phone_no character varying (15)	designation character varying (100)	ifsc character varying (20)

Data Output Messages Notifications

loan_id [PK] character varying (30)	amt double precision	interest double precision	start_date date	duration integer	status character varying (20)	ifsc character varying (20)

Data Output Messages Notifications

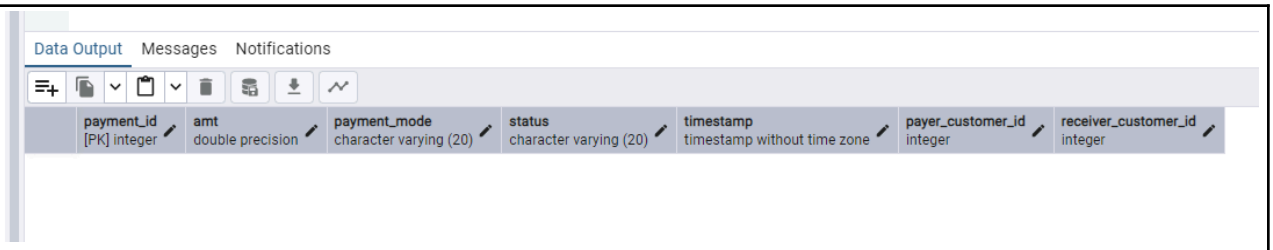
locker_id [PK] character varying (30)	rent double precision	start_date date	duration integer	status character varying (20)	locker_size integer	ifsc character varying (20)

Data Output Messages Notifications

allocation_id [PK] character varying (30)	start_date date	end_date date	status character varying (20)	locker_id character varying (20)

Data Output Messages Notifications

login_id [PK] integer	username character varying (50)	password_hash text	last_login timestamp without time zone	status character varying (20)	customer_id integer




```
CREATE TABLE admin (
 ifsc VARCHAR(20) PRIMARY KEY,
Address VARCHAR(60)
);

CREATE TABLE Accounts (
 ACC_NO VARCHAR(50) PRIMARY KEY,
 BALANCE DOUBLE CHECK (BALANCE > 0),
 Created_At DATE NOT NULL,
 Account_type VARCHAR(20) NOT NULL,
 Status VARCHAR(20),
 ifsc VARCHAR(20),
 CONSTRAINT fk_ifsc
  FOREIGN KEY (ifsc)
  REFERENCES Admin(ifsc)
);



CREATE TABLE Employee (
 EMP_ID INTEGER PRIMARY KEY,
 Salary INTEGER CHECK (Salary > 0),
 Hire_Date DATE NOT NULL,
 Name VARCHAR(100) NOT NULL,
 Department VARCHAR(100) NOT NULL,
 Phone_No VARCHAR(15) UNIQUE,
 Designation VARCHAR(100),
ifsc VARCHAR(20),
 CONSTRAINT fk_ifsc
  FOREIGN KEY (ifsc)
  REFERENCES Admin(ifsc)
);

CREATE TABLE LOANS(
        Loan_ID varchar(30) PRIMARY KEY,
        Amt DOUBLE PRECISION NOT NULL,
        Interest DOUBLE PRECISION,
        Start_date DATE NOT NULL,
        Duration INTEGER NOT NULL,
```

```
            Status VARCHAR(20),
             ifsc VARCHAR(20),
  CONSTRAINT fk_ifsc
    FOREIGN KEY (ifsc)
    REFERENCES Admin(ifsc)
);

CREATE TABLE LOCKER(
            Locker_ID varchar(30) PRIMARY KEY,
            Rent DOUBLE PRECISION NOT NULL,
            Start_date DATE NOT NULL,
            Duration INTEGER NOT NULL,
            Status VARCHAR(20),
            locker_size INTEGER,
             ifsc VARCHAR(20),
  CONSTRAINT fk_ifsc
    FOREIGN KEY (ifsc)
    REFERENCES Admin(ifsc)
);

CREATE TABLE LOCKER_ALLOCATION(
            Allocation_ID varchar(30) PRIMARY KEY,
            Start_date DATE NOT NULL,
            End_date DATE NOT NULL,
            Status VARCHAR(20),
             Locker_ID VARCHAR(20),
  CONSTRAINT fk_lid
    FOREIGN KEY (Locker_ID)
    REFERENCES Locker(Locker_ID)
);

CREATE TABLE customer (
  customer_id INTEGER PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone_no VARCHAR(15) UNIQUE NOT NULL,
  aadhar_no VARCHAR(12) UNIQUE NOT NULL,

  acc_no VARCHAR(50),
  loan_id VARCHAR(50),
  allocation_id VARCHAR(50),

  FOREIGN KEY (acc_no)
    REFERENCES accounts(acc_no),

  FOREIGN KEY (loan_id)
    REFERENCES loans(loan_id),
```

```
  FOREIGN KEY (allocation_id)
    REFERENCES LOCKER_allocation(allocation_id)
);

CREATE TABLE net_banking (
  login_id INTEGER PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  last_login TIMESTAMP,
  status VARCHAR(20),

  customer_id INTEGER NOT NULL,
  CONSTRAINT fk_customer
    FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id)
);

CREATE TABLE payment (
  payment_id INTEGER PRIMARY KEY,
  amt DOUBLE PRECISION NOT NULL CHECK (amt > 0),
  payment_mode VARCHAR(20),
  status VARCHAR(20),
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  payer_customer_id INTEGER NOT NULL,
  receiver_customer_id INTEGER NOT NULL,

  CONSTRAINT fk_payer
    FOREIGN KEY (payer_customer_id)
    REFERENCES customer(customer_id),

  CONSTRAINT fk_receiver
    FOREIGN KEY (receiver_customer_id)
    REFERENCES customer(customer_id),

  CHECK (payer_customer_id <> receiver_customer_id)
);

CREATE TABLE transaction (
  transaction_id INTEGER PRIMARY KEY,
  acc_no VARCHAR(50) NOT NULL,
  type VARCHAR(10) NOT NULL CHECK (type IN ('DEBIT','CREDIT')),
  amt DOUBLE PRECISION NOT NULL CHECK (amt > 0),
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  payment_id INTEGER,
```

```
  netbanking_id INTEGER,

  FOREIGN KEY (acc_no)
    REFERENCES accounts(acc_no),

  FOREIGN KEY (payment_id)
    REFERENCES payment(payment_id),

  FOREIGN KEY (netbanking_id)
    REFERENCES net_banking(login_id)
);
```

Conclusion:

This experiment successfully implemented table creation using SQL.

Post Lab Questions:

1. **Which command is used for removing a table and all its data from the database:**

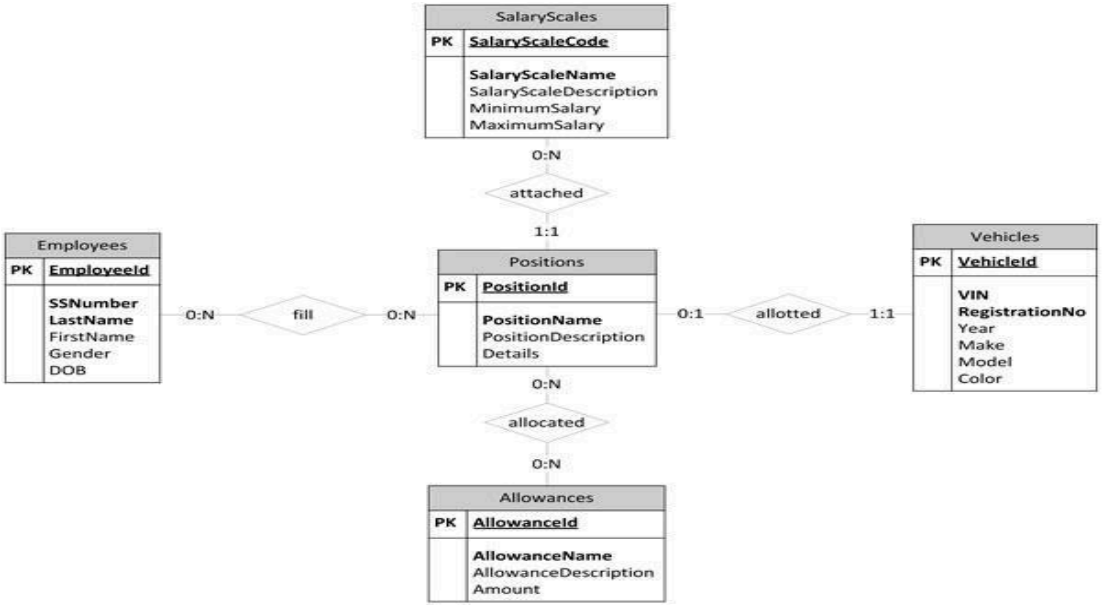
   A. DROP Command
   B. TRUNCATE Command
   C. Both Commands

Ans: DROP command

2. **For the given ER model, using DDL command: Write syntax to create CREATE Tables with all possible integrity constraint**

```
CREATE TABLE position (
  position_id INTEGER PRIMARY KEY,
  title VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE salary_scale (
  scale_id INTEGER PRIMARY KEY,
  min_salary DOUBLE PRECISION NOT NULL,
  max_salary DOUBLE PRECISION NOT NULL,
  position_id INTEGER UNIQUE NOT NULL,
  FOREIGN KEY (position_id) REFERENCES position(position_id)
);

CREATE TABLE allowance (
  allowance_id INTEGER PRIMARY KEY,
```

```
  allowance_name VARCHAR(50),
  allowance_description VARCHAR(200),
  amount DOUBLE PRECISION NOT NULL,
  position_id INTEGER NOT NULL,
  FOREIGN KEY (position_id) REFERENCES position(position_id)
);


CREATE TABLE employeePL (
  emp_id INTEGER PRIMARY KEY,
  SS_NO BIGINT NOT NULL,
  lastname VARCHAR(100),
  firstname VARCHAR(100) NOT NULL,
  gender VARCHAR(100) NOT NULL,
  DOB DATE NOT NULL,
  position_id INTEGER NOT NULL,
  FOREIGN KEY (position_id) REFERENCES position(position_id)
);

CREATE TABLE vehicle (
  vehicle_id INTEGER PRIMARY KEY,
  REG_no VARCHAR(20) UNIQUE,
  model VARCHAR(50),
  Year INTEGER,
  make varchar(50),
  colour varchar(50)
);

CREATE TABLE vehicle_assignment (
  assignment_id INTEGER PRIMARY KEY,
  vehicle_id INTEGER UNIQUE,
  position_id INTEGER UNIQUE,
  FOREIGN KEY (vehicle_id) REFERENCES vehicle(vehicle_id),
  FOREIGN KEY (position_id) REFERENCES position(position_id)
);
```

Problem Statement:

A small accounting firm wants a simple HR application that will help it to keep track of its employees, their positions, allowances, salary scales, and which company vehicles their employees drive. The application must keep track of all the positions at the firm, the employees filling these positions, the allowances for these positions, the salary scales for these positions, and the company vehicles assigned to these positions.